

# COS4070 - Project 1

Send in the Rogues

Version 1.0

Z. Hutchinson

zhutchinson@aubg.edu

September 6, 2026

## 1 Due Date

A final submission is due: **Oct 16th @ Midnight.**

**This is a hard deadline.** Any submissions after midnight will receive a 0. You have a long time to work on this project. Do not ask for an extension.

## 2 Project Files

You can find the project 1 code here:

[https://github.com/zaxhutchinson/Microworld2\\_VS.git](https://github.com/zaxhutchinson/Microworld2_VS.git).

The directory contains a working makefile. Provided *make* is in the path of your OS, typing 'make' should build the project. 'make clean' will remove all object files and the executable.

## 3 Goals

The primary goal of this project is to give you the experience of working on a modestly complex agent-based AI task. This goal is achieved if you:

- Feel overwhelmed by the many possible, albeit imperfect, solutions.
- Make peace with failure.
- Feel happy when ideas work some of the time.
- Get excited when something unexpected happens.
- Realize programming common sense knowledge is very hard.
- Get to a point where you have no idea how to improve your solution.

Therefore, do not be dismayed by failure. Keep trying. Be prepared to start over several times in whole or part.

AI, like biological intelligence, is created by a journey with many ups and downs. It is not created in a day, born perfect in every way.

## 4 General Instructions

Read this document fully and carefully. The devil is in the details. If something is unclear either ask in class or stop by my office.

For this task, there is a dungeon full of traps and treasures. Into this dungeon are sent one or more rogues whose task is to find all the treasure. Treasure is visible and there for the taking. Traps, however, are hidden but your rogues are clever. Each of them can sense how close the nearest trap is to their current location.

Your task is to devise an AI that will enable the rogues to take all the treasure while also locating and disarming any hidden traps in their way. The environment is modestly complex consisting of obstacles that limit movement and vision. It also consists of teleporters that move your rogues around the map. A full account of the rogues and environment are provided below.

If you find any bugs that either crash the program or deviate from the description below, please email me right away. I will fix the bug and upload a new version.

## 5 Solution Restrictions

### 5.1 ai.hpp and ai.cpp

Your entire solution must be within the ai.hpp and ai.cpp files. See submission section.

### 5.2 Other Modifications

You may not modify any other simulator files. It won't help you anyway. I will use my own copies of the other files when grading.

You may not `#include` additional simulator files into the ai files except what are already there. You are welcome to include all the standard library files you like.

### 5.3 Limited Information

Your AI may not load the world file itself. For world information, it must rely on what it receives through the `Percepts` struct. Likewise, rogue actions must rely solely on returned command strings.

Also, AI's may not try to gain access to other parts of the simulation, which includes other AIs. This program is meant to model individual agents who cannot directly access each other's memory.

### 5.4 C++ Standard and Additional C++ Libraries

I intend to grade using the C++17 standard. If you utilize additions beyond this standard, your program will be considered incorrect (as if it had a bug...which technically it does).

Your solution should use only what comes with a standard C++17 installation. **I will not install additional libs to make your code work.** If you need code, write it.

### 5.5 What is Not Restricted

You are welcome to modify and add to the AI class. There are no restrictions against keeping state within the AI object. In fact, this is encouraged (otherwise you have a reactive agent). The only requirements are that you do not modify the parameter lists, names and return values of the functions expected by the rest of the simulator (e.g., constructor and `RunAI`).

## 6 Environment

The environment is a 2D grid-based map consisting primarily of walls and open spaces. Additionally, open spaces can contain one of several special properties. These are all described below.

The environment is generated from a text file found with the simulator. Different characters in the file specify different types of map cells.

List of cell characters:

- w is a wall. Walls are the only impassable cell type.
- o is an open space without any special properties.
- @ is a starting location for rogues.
- ] [ ( ) { } are all teleporter cells.
- ^ is a trap.
- # is an exploded trap.
- + is a disarmed trap.
- \$ is a treasure.

Only rogue starting locations and hidden traps are invisible in the rogue's percepts. Rogues themselves are also completely invisible in the environment which means they cannot see each other. However, they do receive relative directions and distances to all other rogues. [Somehow all rogues know the relative position of all other rogues.]

Several test environments are provided with the simulator. All world files should be stored in the worlds directory. Each world file is accompanied by a config file with the same name. This file gives the intended configuration for this specific world. You should set the corresponding values in the config.json. For example, some worlds are intended to use multiple rogues. Some have different speed values for the rogues. Some have different teleporter schemes, etc. Just copy the contents of the world config to config.json (or replace and rename it).

You are free to make your own to further test your AI (this is encouraged). **If you do, please note that all environments must contain wall cells along the exterior of the map.**

## 6.1 Special Cells

### *Teleporters*

If an agent uses a teleporter, it will be instantly moved to another location on the map. There are six possible teleporter symbols: ] [ ( ) { }. Not all of them have to appear on the map, but there will be no more than six. They can be linked to each other in any way. This means that using a [ teleporter might take you to the ) teleporter. However, using the ) teleporter might take you to the ( teleporter and not back to the [. A teleporter can even link to itself. They are given distinct symbols so that your AIs can learn which symbols link to which other symbols. Teleporters can be used repeatedly.

For regular movement, teleporter cells are treated just like open cells.

### *Traps*

Traps are dangerous things within the environment, which, if not disarmed will destroy the unwary rogue moving into them. While traps appear on the graphical display of the environment, they do not appear in rogue percept lists. To the rogue, a trapped cell looks like an open cell. If a rogue moves into a trapped cell, it is destroyed.

To disarm a trap, the rogue must be in an adjacent cell and facing the trapped cell and issue a 'D' command. This will disarm the trap.

Exploded traps and disarmed traps are visible to the rogue. They are visible to allow the rogue to know whether or not it has been successful. Once disarmed or exploded, a trap is harmless and for movement is treated like an open cell.

### *Treasures*

Treasures are the goal of the game. Rogues (of course) want treasure. They risk life and limb to get it. Treasures appear on the map as a \$. To collect a treasure, a rogue must stand on top of a treasure and emit the 'T' command. This will cause the rogue to pick up the treasure and the cell where the treasure was is replaced by an open space: 'o'.

NOTE: Treasures and traps cannot occupy the same location.

### Starting Locations

Starting locations are used only to place agents at predefined locations on the map. Once the simulation starts, starting locations are invisible to agents. They also cannot be seen in the display. NOTE: if there are more agents than starting locations, some will start in the same cell.

## 6.2 Percepts

The simulation runs rogue-by-rogue. Each turn all rogues still active get a chance to move. At the start of its move each rogue is provided with a *percepts* object (see *percepts.hpp* file for details). This object contains what the rogue can see or sense. The percept object contains seven data points. It contains five vectors of single character strings corresponding to cell contents for all directions it can see. There is a vector for what is in front of the rogue, behind, left and right. There is also a vector for the cell underneath the rogue; however, this vector always has only one string.

The contents of each of the four directional percept vectors (forward, backward, left and right) is limited by one of two things: walls and sight limitations. Vision extends until a wall is encountered (sight cannot pass through walls) or until the rogue's max sight is reached.

The sixth data member of the percepts object, *detector*, is an integer giving the *Manhattan distance* to the nearest trap. NOTE: this is calculated once per turn. If an rogue speed's is greater than one, be mindful that rogues do not have access to an updated trap distance to decided multiple actions within the same turn. **If there are no more traps in the environment, this value will show -1.**

The seventh data member, *others*, contains a list of 2d direction vectors to all other rogues (include the rogue itself...which would be 0,0). While rogues cannot see each other they can detect each other's relative direction. Directions are relative to a) the location of the rogue and b) its heading. Positive X values are to the right of. Negative X is to the left. Positive Y values are in front of. Negative Y is behind. For example, a direction of 2,-7 means that another rogue is 7 cells behind it and 2 cells to the right.

## 7 Commands

Each turn, each rogue's RunAI must return a vector of single character strings. Each single character string is a command.

Which commands to issue? Well, that's the whole problem. The vector can be empty, in which case, the rogue does nothing. There are five possible commands:

- F: move forward one cell
- B: move backward one cell
- L: turn left (but stay in same cell)
- R: turn right (but stay in same cell)
- U: use current cell (for teleporters)
- D: disarm the cell in front of the rogue
- T: take the treasure in the current cell

### 7.1 Command Notes

To use a teleporter a rogue must stand on it and issue the 'U' command. Just walking over a teleporter does not activate it.

To collect a treasure a rogue must stand on the treasure and issue the 'T' command. Just walking over the treasure does not collect it.

'L' and 'R' turn (i.e., rotate) the rogue's body. They do not strafe.

'D' requires both proper placement and facing to be successful.

## 8 Movement and Turning

Each rogue has a facing within the environment. The rogue will never know precisely which way is north or south. It does not know its own absolute orientation. However, the simulation keeps track of the rogue's absolute heading. If an rogue is facing south and moves forward, it will move one cell to the south. If it turns left, it will now be facing east.

For example, if you wanted the rogue to turn right, your vector would contain a single "R".

## 9 Speed

Rogues have a speed. Speed indicates how many commands can be returned from the RunAI function. Each one will be executed in the order they are added to the vector. For example, were an rogue with three speed to return a vector containing "L", "F" and "U" the simulation would cause it to turn left, move forward one cell and then use the cell (provided the rogue type could do these things).

If an rogue returns more commands than its speed allows, the additional commands are ignored. An rogue can return fewer commands than its speed allows.

In the default setup, rogues have a speed of one. However, I encourage you to experiment with different values once your AI routines are more advanced.

## 10 Sight

Rogues have sight limitations in the four cardinal directions. For example, a rogue with Forward 6 can see the 6 cells in front of it. The contents of these six cells will be included in the percepts object. However, walls impede vision. Although a rogue might be able to see 6 cells, if it is facing a wall that is two cells away, the forward vector will only contain three strings (two open cells and a wall).

## 11 Environment and Agent Variables

The AIs are provided a Symbols struct that contains the symbols used for different things in the environment. I advise you write your AI based on these rather than hard-coding the string matching.

AIs are also provided the rogue's speed. This should be used by the AI to know how many commands it can issue.

AIs have access to the number of rogues in the environment in the AgentComms struct (see below). NOTE: This number does not change when an rogue is destroyed.

## 12 Rogue Communication

Rogues have the ability to communicate with each other. The RunAI method in the AI, takes a second parameter, a pointer to an *AgentComm* object. You can find the declaration of the *AgentComm* class in the *comm.hpp* file.

NOTE: you are not allowed to alter the *comm.hpp* file or the *AgentComm* class.

The *AgentComm* class contains an integer giving how many agents started in the environment and an array of unsigned integers to be used for communication.

During the simulation, there is only one instance of the *AgentComm* class shared by all rogues. The class contains an array of unsigned 32-bit (most likely) integers. The size of the array is fixed according to the number of rogues. You may use these unsigned integers as you wish to encode communication between rogues (if there are more than one rogue). Keep in mind that each AI is given its rogue's id. Rogue ids start from 0 and count up. If there are 3 rogues in the environment, the rogue ids will be 0, 1 and 2. You can use these to index the comm's signals array. This is only one idea.

NOTE: If an rogue is destroyed, it does not have an opportunity to communicate this fact.

## 13 Slip

You will find a value in the config.json (I believe it is the last value) called *slip\_chance*. If set to a non-zero value, this is the chance that a rogue's commands are not carried out. This impacts all actions.

By default, this is set to zero point zero. All actions always work.

When this is set to a non-zero value, the rogue cannot be sure that what it tried to do previously actually worked. This opens quite a few new concerns as the rogue must first verify that what it tried to do actually happened. For example, let's say it issued an "F" previously. If it assumes it moved, then integrating new percepts into its internal map of the environment will screw up the map.

I consider have added this extra challenge for anyone who wants it. If you try to account for slippage, please indicate this in your report and indicate the percent you used. I advice a low value (less than five percent).

You should also set the agent speed to 1.

## 14 Scoring

Every action has a cost. Most costs are negative and some (well, one) is positive. Costs are defined in the config.json file, but I will duplicate them here.

- 0 for every round (turn).
- -1 for moving forward or backward.
- -1 for turning left or right.
- -5 for using a teleporter.
- -10 for an unsuccessful disarm attempt (i.e., there was no trap).
- -10 for an unsuccessful teleport use (i.e., there was no teleporter).
- -10 for running into a wall (moving forward or backward).
- -100 for destroying the agent on a trap.
- +100 for disarming a trap.
- +1000 for collecting a treasure.
- -1 for doing nothing in a turn.

A few things to note. Doing anything (including nothing) costs points. Making poor moves, such as running into a wall or trying to take a treasure that isn't there costs more points. Your rogues are only rewarded (in this default setup) for disarming traps and collecting treasure.

For your own benchmarking, you can alter these values to penalize different types of movement. When grading I might use a different point scheme than the default one. Or I might use several.

NOTE: Points have no connection to the grade. Points exist for you and me to rate two different solutions in a given environment. It is possible (and even likely) that A-level solutions will have a low score.

## 15 Starting Locations

Rogue starting locations (and headings) will be randomized for grading.

## 16 The End

There are three ways the simulation ends.

- 1) If all rogues are destroyed.

- 2) If all treasures are collected.
- 2) If the max turn limit is reached.

## 17 Grading

The points breakdown is as follows: 60% solution and 40% report.

**Grading will be done using one or more environment (i.e. map) not included with the project files.** This means your AI implementation cannot "hard-code" environment specific solutions.

Rogue speed, sight limitations, teleporter links, number of rogues, number of traps, number of teleporters, size of the map, number of starting locations could all change between the environments I use for grading. I enjoy throwing edge cases at your AIs. I advise you not to base your solution around one specific set up.

### 17.1 Grading the Solution

Here are a list of issues that can cause you to lose points. This list is not exhaustive. This list does not have points attached. The number of points will be assigned according to the severity of the issue. In general, you need to make a strong effort to implement the AI and convince me of your work. You should be able to verbalize your approach. Lazy solutions, above all else, will not score well.

- Crash (program ending bug).
- Incomplete or unfinished AI.
- AI implementation does not attempt to solve the problem (purely random solutions, phony solutions, obviously bad solutions, etc.).
- No good use of the AgentComm data (the agents don't talk to each other).
- AI does not make use of rogue distances.
- AI does not make use of trap distances.
- AI does not map the environment.
- AI does not make use of all percepts.
- AI does not return the full range of commands or they are returned randomly or in the wrong context.
- Code readability (for the case I am forced to look at the code).
- Yes, you will be graded on how well the rogues work, but you will also be graded on the 'interestingness' of your solution.
- Debug or log output left enabled.
- Altering the simulation to circumvent the restrictions (expect to fail).

### 17.2 Grading the Report

The report is a very important component of this project. That is why it is worth 40 percent of your grade. In the real world (i.e., outside school) no one cares about your code. If you want someone to take interest in what you've done, you will need to put it in a form everyone can digest.

The report should be a professional-looking **PDF** that contains the following sections:

- 1) A detailed, well-written, English description that could be understood by someone without a CS background. You are free to frame the description as you wish but it should be comprehensive. What is comprehensive? Well, I will read your report first and then I watch your rogues solve the task several times. I should be able to see your characterizations of the solutions to subproblems in the behavior of your rogues.

2) A comprehensive discussion of all behavior-level problems your final AI has. Behavior-level is not code-level. A behavior-level problem is description of different situations in which the rogue fails to perform. Describe not just that it fails, but what are the conditions. Failure to perform itself is not a binary thing. For example, the rogue might get stuck in a corner only if a trap is nearby; otherwise it won't get stuck. Try to analyze the failures and describe them. Remember the goals of this project: 'make peace with failure'.

3) A meaningful after-action report on what it was like for you to craft your solution. Think of this as the story of the work. Where did you go wrong? Where did you go right? What helped you to find the solution? What hurt you? These are a few prompts to get you started but don't let them limit your story of the project.

4) You are welcome to include anything else you think relevant: diagrams, pictures, sketches, whatever, but the report isn't a dumping ground. You don't get points by the page. If I feel you've filled the report with unedited notes, scratchings or other mid-project text in the hopes of getting a good grade,...nope.

Again, I warn a lazy report will suffer. Don't leave this to the last minute. Take notes as you go. Document changes in a change log (or some other log method). Keep a notebook of ideas. Use these to craft your report.

## 18 Submission Instructions

Submit to Canvas a zip file with three files: *ai.hpp*, *ai.cpp* and your report in PDF format.

## 19 Installation and Running

### 19.1 Installation

This project requires that you have SFML-3.0.0 installed. Here is a link to instructions if you are using Windows and Visual Studio. [SFML Installation Instructions](#) On this site, there are also instructions for Linux, CodeBlocks, and Mac. If you miss it or are having trouble, **please stop by my office!**

### 19.2 Running

Microworld2 supports a number of command line arguments.

- `-t <int>` The `-t` argument followed by an integer specifies the max turns a simulation will run. If max turns are reached, the simulation ends and current points are displayed.
- `-d <int>` The `-d` argument followed by an integer specifies in milliseconds the time between AI updates. For example, `-d 1000` places a 1 second delay between agents. Lower values will speed up the simulation. Higher values will allow you to examine what is happening. The default is 1000.
- `-w <string>` The `-w` argument allows you to specify different world files to use. For example, `-w world1` would load the simulation from a file *world1*. If `-w` is omitted a file called *default\_world* is looked for. All world files are expected to be in the *worlds* subdirectory of the main directory.
- `-rs` The `-rs` argument enables random start positions.
- `-rh` The `-rh` argument enables random headings for all agents. If `-rh` is omitted, all agents begin the simulation facing north.
- `-rsh` (or `-rhs`) This argument (and its variant) enable both randomized starting locations and randomized headings. **This argument will be used for grading.**
- `-s <unsigned int>` The `-s` argument seeds the simulator's random number generator with the unsigned int provided. This allows you to use the random arguments above but replay runs with deterministic behavior. The random number generator is also passed by pointer into each type of AI and stored as a private member variable. If you base any randomness within your AI routines on this rng variable, they will also be deterministic when the seed is set. Can be handy for testing. Omitting this argument causes the simulator to use `std::random_device` to seed the rng.

## 19.3 Hotkeys

**E**scape will immediately terminate the simulation without displaying the final score.

**Q** will finish the simulation displaying the current results.

**P** will pause the sim (if you want to study some aspect of it). Pressing P when it is paused will unpause it.

## 20 Advice

Start small. First, use pencil and paper to plan some aspect of the AI. Think of the different micro scenarios that might happen. What should the agent do in that situation?

Do not hard-code values from the config files. For example, don't assume there will only be one rogue, or that rogue sight has some specific distance, or that teleporters work a certain way. Of course, some things won't change (e.g., the symbols). However, I suggest you use the Symbols and Costs structs in your AI code.

Next, how will agents explore the map? How will they use pathfinding to go from one location to another? How will they store the map as they explore it? Work on support code (functions) that perform common tasks.

Consider using concepts like belief and uncertainty to guide agents in meta ways. If they are mapping the environment and keeping track of what they've seen, can they logically conclude certain things? Where is it safe? Should it relocate? Finding ways to (sometimes) utilize logically acquired meta knowledge is what separates a reactive, simple solution from a thoughtful, complex one.

Use simple maps to test your solutions. Rearrange the map to set up different scenarios. Then move to larger maps. Test, test, test.

While purely random solutions are not acceptable, your solution should use randomness in some way (of course, it does not have to). Random solutions allow agents to perform more toward the average on any given map. Deterministic solutions are very dependent on a) the map and b) where they start.

Remember percepts are collected *rogue-by-rogue* rather than once for all each turn. What does this mean? This means that when it is a rogue's turn to act (when the AI code runs), what it sees is the actual state of the world.<sup>1</sup>

What rogues say to each other via the unsigned integers in the AgentComms struct is completely up to you. But communication is tricky. Remember, a rogue will alter any communication values **before** the simulation moves it. Because of this, the other rogue's percepts could contain information from two different points in time as the communication it receives will have been created at its previous location. The rogue's communication and the rogue's subsequent location will not line up. If you wish to have other rogues combine such information, it might wise to a) communicate it or b) not move when communicating.

Have fun. AI is about trying to solve hard problems with risky (e.g, high probability of not working), avant garde ideas.

---

<sup>1</sup>This is in contrast to turn-by-turn where all information is collected at the start of the turn for all agents and all actions are executed at the end of a turn. Depending on a rogue's order within the turn, the world might have changed.